100-days-of-python/

│

├── README.md # Main repo overview

├── .gitignore

│

├── day-01/

│ ├── main.py

│ └── README.md

│

├── day-02/

│ ├── main.py

│ └── README.md

│

├── day-03/

│ ├── main.py

│ └── README.md

│

├── ...

│

├── day-100/

│ ├── main.py

│ └── README.md

│

└── utils/ # optional (later refactors)

└── helpers.py

 # 100 Days of Python 🚀

This repository documents my 100-day journey to become confident and fluent in Python programming.

🎯 Goals

- Build strong Python fundamentals

- Write clean, readable, and maintainable code

- Learn by building small daily projects

- Be interview-ready with clear explanations

🛠️ Tech Stack

- Python 3.x

- CLI-based projects

- No frameworks (focus on fundamentals)

📁 Structure

Each folder represents one day and one project.

Day XX – Project Name

📌 Problem Statement

Briefly describe what the program does and why it is useful.

🎯 Objectives

- What I wanted to learn

- Which Python concepts I practiced

🛠️ Concepts Used

- Variables

- Conditionals

- Loops

- Functions

- Lists / Dictionaries (as applicable)

🧠 Approach

Explain your logic in simple steps:

1. Took input from user

2. Validated input

3. Processed data

4. Displayed formatted output

⏱ Time & Space Complexity

- Time Complexity: O(n) / O(1) (explain briefly)

- Space Complexity: O(n) / O(1)

🧹 Clean Code Practices

- Used meaningful variable names

- Avoided hard-coded values

- Kept logic inside functions

- Followed single responsibility principle

🔍 Possible Improvements

- Add file storage

- Add validation

- Add UI / Web interface

🧪 Sample Output

SCRIPT: “Explain One of Your Python Projects”

Step 1: Context (10 seconds)

> **“This was a small Python CLI project I built as part of a 100-day learning challenge to strengthen my fundamentals.”**

Step 2: Problem (15 seconds)

> **“The goal was to build a program that takes user input, processes it logically, and outputs structured results.”**

Step 3: Design Decisions (30 seconds)

> **“I structured the code using functions to keep logic modular and readable.**
> **I avoided global variables and used clear naming to improve maintainability.”**

Step 4: Optimization & Complexity (20 seconds)

> **“The solution runs in linear time since it processes each input once.**
> **Memory usage is minimal as no unnecessary data structures are used.”**

Step 5: Clean Code Defense (30 seconds)

> **“I focused on clean code by keeping functions small, avoiding repetition, and making the logic easy to follow.**
> **This makes the code easier to test and extend.”**

Step 6: Scalability (15 seconds)

> **“If needed, this could be extended by adding file storage, better validation, or a UI without changing the core logic.”**

❓ COMMON FOLLOW-UP QUESTIONS (WITH STRONG ANSWERS)

❓ Why is your code clean?

> **"It's readable, modular, avoids duplication, and each function has a single responsibility."**

❓ Why didn't you use classes here?

> **"The problem didn't require shared state or complex behavior. Functions were simpler and more appropriate."**

❓ How would you improve this?

> **"I'd add persistent storage, logging, and unit tests."**

❓ What did you learn from this?

> **"It helped me think in terms of logic, structure, and edge cases rather than just syntax."**

5️⃣ COMMIT MESSAGE TEMPLATE (IMPORTANT)

Use consistent commits:

```
feat: day-23 password strength checker using string analysis
```

or

```
refactor: improve readability and remove duplicate logic
```

This shows professional Git hygiene.

✅ FINAL CHECKLIST (PIN THIS)

Before pushing ANY project:

- **Code runs without errors**
- **Variables are meaningful**
- **Logic is inside functions**
- **README explains reasoning**
- **Commit message is clear**

1️⃣ EXACT FOLDER + FILE NAMING RULES

These rules make your repo look clean, predictable, and professional.

📁 Folder Naming Rules

Daily folders

```
day-01/

day-02/

...

day-09/

day-10/
```

✅ Always use two digits

❌ Never: `day1`, `Day-1`, `day_one`

📄 File Naming Rules

Main program

`main.py`

Why?

- **Everyone knows where execution starts**
- **Interviewers love convention**

Helper / utility files (only when needed)

`helpers.py`

`validators.py`

`storage.py`

Rules:

- **snake_case**
- **Nouns or verbs that describe responsibility**
- **Avoid `utils.py` early (too vague)**

Test files

`test_main.py`

```
test_helpers.py
```

📦 Imports Rule

Always absolute imports within the day folder.

Example:

```
from helpers import validate_age
```

🗒️ Naming Inside Code

Variables

```
user_age      # good

x             # bad
```

Functions

```
calculate_total()

is_valid_input()
```

Constants

```
MAX_ATTEMPTS = 3
```

2️⃣🧪 HOW TO WRITE SIMPLE TESTS (BEGINNER FRIENDLY)

You do NOT need advanced testing.

🔹 Rule: Test logic, not input()

Bad:

```
input()
```

Good:

```
def is_even(n):
    return n % 2 == 0
```

🧪 Minimal Test Example

`test_main.py`

```
from main import is_even


def test_even_number():
    assert is_even(4) == True
```

```
def test_odd_number():

    assert is_even(3) == False
```

Run:

```
python -m pytest
```

🧠 Interview Explanation

> **“I separated logic from input so it can be tested independently.”**

This line alone = 🚀

✅ Testing Checklist

- **No input() inside tested functions**
- **Clear expected outputs**
- **Covers edge cases**

3️⃣🔁 HOW TO REFACTOR OLD DAYS (THIS IS HUGE)

Refactoring shows growth.

🧹 Refactor Rules

Step 1: Identify problems

- **Repeated code**
- **Long functions**
- **Magic numbers**

Step 2: Extract functions

Before:

```
if age >= 18:

    print("Adult")
```

After:

```
def get_age_category(age):

    return "Adult" if age >= 18 else "Minor"
```

Step 3: Add README Improvements

Add section:

```
## Refactoring Notes

- Extracted logic into functions

- Improved naming

- Reduced duplication
```

Step 4: Commit Properly

```
refactor: improve modularity and readability for day-05
```

🧠 Interview Gold Line

> **“I revisited earlier projects to refactor them using better practices as my understanding improved.”**

4️⃣🎯 HOW TO PRESENT THIS REPO IN RESUME

This is how you turn practice → proof.

📌 Resume Project Entry (COPY THIS)

100 Days of Python – Project-Based Learning

GitHub: github.com/yourname/100-days-of-python

- **Built 100 Python mini-projects covering fundamentals, data structures, file handling, and OOP**
- **Focused on clean code, modular design, and testing**
- **Wrote daily documentation explaining design decisions and complexity**
- **Refactored older projects to improve maintainability and readability**
- **Used Git with meaningful commits to track progress**

🎤 How to Explain in Interview

“Instead of only solving problems, I focused on building small projects daily and documenting my design decisions.
This helped me become comfortable writing and explaining Python code.”

❌ What NOT to Say

❌ **“I followed a tutorial”**
❌ **“I copied projects”**

✅ What TO Emphasize

- **Consistency**
- **Refactoring**
- **Testing**
- **Design thinking**

🔥 FINAL INTERVIEW CHECKLIST

Before interviews:

- **Pick 3 favorite days**
- **Be able to explain them in 2 minutes each**
- **Know one refactoring story**
- **Know one bug you fixed**

🐍 100 DAYS OF PYTHON — DAILY PROJECT IDEAS

🧱 DAYS 1–10: Absolute Python Basics (Syntax Confidence)

Goal: Stop thinking about syntax

Day 1 – Print-based Introduction App
→ Prints name, age, goal using variables

Day 2 – Simple Calculator
→ Add, subtract, multiply, divide two numbers

Day 3 – Age Category Checker
→ Child / Teen / Adult

Day 4 – Even or Odd Analyzer
→ Takes number, prints result

Day 5 – Number Guessing Game (Fixed Number)
→ User guesses until correct

Day 6 – Temperature Converter
→ Celsius ↔ Fahrenheit

Day 7 – Multiplication Table Generator
→ For any number

Day 8 – Simple Interest Calculator

Day 9 – Grade Calculator
→ Based on marks

Day 10 – CLI Welcome Menu
→ Menu-based program with options

✅ ***You're good to move on if indentation feels natural.***

🔁 DAYS 11–20: Loops + Logic Building

Goal: Think in loops

Day 11 – Sum of First N Numbers

Day 12 – Count Digits in a Number

Day 13 – Reverse a Number

Day 14 – Palindrome Number Checker

Day 15 – Prime Number Checker

Day 16 – Print All Primes Till N

Day 17 – Fibonacci Series Generator

Day 18 – Pattern Printer (Stars – Pyramid)

Day 19 – Pattern Printer (Numbers)

Day 20 – Menu-Based Math Tool

🧵 DAYS 21–30: Strings Mastery

Goal: Comfort with indexing & slicing

Day 21 – Reverse a String

Day 22 – Palindrome String Checker

Day 23 – Vowel & Consonant Counter

Day 24 – Word Count Tool

Day 25 – Longest Word Finder

Day 26 – Password Strength Checker

Day 27 – Remove Duplicate Characters

Day 28 – Anagram Checker

Day 29 – String Compression Tool

Day 30 – Sentence Formatter

📦 DAYS 31–40: Lists & Arrays

Goal: Data handling mindset

Day 31 – Sum of List Elements

Day 32 – Max & Min in List

Day 33 – Remove Duplicates from List

Day 34 – Rotate List

Day 35 – Second Largest Element

Day 36 – Frequency Counter

Day 37 – Merge Two Lists

Day 38 – Find Missing Number

Day 39 – Sort Numbers Without sort()

Day 40 – Simple Student Marks Manager

🗂️ DAYS 41–50: Dictionaries & Hashing

Goal: Think in key–value pairs

Day 41 – Phone Book App

Day 42 – Word Frequency Counter

Day 43 – Expense Tracker

Day 44 – Inventory Management System

Day 45 – Voting System

Day 46 – Student Grade Book

Day 47 – Frequency-Based Sorting

Day 48 – Simple Cache Simulator

Day 49 – Character Frequency Visualizer

Day 50 – Quiz App (Questions stored in dict)

🧠 DAYS 51–60: Functions & Recursion

Goal: Modular thinking

Day 51 – Calculator Using Functions

Day 52 – Prime Check Using Function

Day 53 – Factorial (Recursive & Iterative)

Day 54 – Fibonacci Using Recursion

Day 55 – Sum of Array (Recursive)

Day 56 – String Reverse (Recursive)

Day 57 – Menu-Based Utility App

Day 58 – Code Refactor Day
→ Refactor old projects into functions

Day 59 – Input Validation Library

Day 60 – Math Utility Module

📂 DAYS 61–70: File Handling & Errors

Goal: Real-world coding

Day 61 – Write Data to File

Day 62 – Read from File

Day 63 – Append Logs

Day 64 – File-Based Notes App

Day 65 – Word Counter from File

Day 66 – Error-Handled Calculator

Day 67 – Safe Login System

Day 68 – CSV Data Reader

Day 69 – File Backup Tool

Day 70 – Mini Text-Based Database

🧩 DAYS 71–80: OOP (Interview-Relevant)

Goal: Clean architecture

Day 71 – Class: Person

Day 72 – Class: Student

Day 73 – Bank Account Simulator

Day 74 – ATM System

Day 75 – Library Management System

Day 76 – E-Commerce Cart

Day 77 – User Authentication System

Day 78 – Inheritance Demo Project

Day 79 – Refactor Old Project into OOP

Day 80 – OOP-Based Task Manager

🚀 DAYS 81–90: Advanced Python Comfort

Goal: Confidence & speed

Day 81 – To-Do App with File Storage

Day 82 – Password Manager

Day 83 – URL Shortener (Logic Only)

Day 84 – Simple Logger Utility

Day 85 – Contact Management System

Day 86 – Quiz Game with Scoreboard

Day 87 – Chat Simulator (CLI)

Day 88 – Mini Search Engine (Text-based)

Day 89 – JSON-Based Data Store

Day 90 – Refactor + Optimize Old Code

🏁 DAYS 91–100: Portfolio & Interview Mode

Goal: Be interview-ready

Day 91 – Final Project Planning

Day 92 – CLI Task Manager (Core Logic)

Day 93 – Add File Persistence

Day 94 – Add Error Handling

Day 95 – Add OOP Structure

Day 96 – Write README (Explain Design)

Day 97 – Code Cleanup & Optimization

Day 98 – Mock Interview Explanation
→ Explain your code out loud

Day 99 – GitHub Polish Day

Day 100 – 🎉 Final Review & Confidence Day

✅ HOW YOU KNOW THIS WORKED

By Day 100:

- **You don’t google basic syntax**
- **You structure code naturally**
- **You can answer:**

    - **“Why did you design it this way?”**
    - **“How would you optimize this?”**
    - **“How would you scale this?”**
- **Your GitHub looks 🔥**

🐍 PYTHON BY DOING — PROJECT ROADMAP

📅 Start date: Feb 10

Total duration: 6–7 weeks
Daily time: 1.5–2.5 hours

WEEK 1 (Feb 10 – Feb 16)

🧱 Python Basics + Control Flow

Topics

- **Variables & data types**
- **Input / output**
- **if / elif / else**
- **Basic operators**
- **Simple loops**

🧪 Project 1: CLI Personal Profile Generator

What to build

A command-line program that:

- **Takes user name, age, country**
- **Classifies user (Child / Teen / Adult)**
- **Prints a formatted profile summary**

Example:

```
Name: Alex
Age: 21
Category: Adult
```

```
Country: India
```

Python Concepts Used

- `input()`, `print()`
- `int()`, strings
- if–else logic

Git Commit Message

```
feat: add CLI personal profile generator using basic python syntax
```

Clean Code Checklist (Interview Gold)

✅ Meaningful variable names
✅ No magic numbers
✅ Proper indentation
✅ Clear separation of input & logic
✅ Readable output formatting

Interview Questions You Can Answer

- Why did you use if-elif instead of multiple ifs?
- How would you validate invalid age input?
- How can this be extended without rewriting code?

➡️ Outcome: You stop fearing Python syntax.

WEEK 2 (Feb 17 – Feb 23)

🔁 Loops, Strings & Lists

Topics

- `for`, `while`
- String slicing
- List operations

- **Basic validation logic**

🧪 Project 2: Password Strength Checker

What to build

A program that:

- **Takes password input**
- **Checks:**
  - **Length**
  - **Digits**
  - **Uppercase**
  - **Special characters**
- **Returns strength: Weak / Medium / Strong**

Concepts Used

- **Loops**
- **String methods**
- **Conditions**
- **Counters**

Clean Code Checklist

✅ Functions for each check
✅ No repeated logic
✅ Early returns
✅ Descriptive function names

Interview Questions

- **Why did you split logic into functions?**
- **What is the time complexity?**

- **How would you improve security?**

➡️ **Outcome: You start *thinking in logic*, not syntax.**

WEEK 3 (Feb 24 – Mar 2)

📦 **Lists, Dictionaries & Hashing**

🧪 **Project 3: Expense Tracker (CLI)**

What to build

- **Add expenses (date, amount, category)**
- **Show total spent**
- **Category-wise summary**

Example:

```
Food: 1200
Travel: 800
```

Concepts Used

- `list` **of** `dict`
- **Aggregation logic**
- **Basic reports**

Clean Code Checklist

✅ **Data structure choice justified**
✅ **Single responsibility functions**
✅ **Easy to extend categories**
✅ **No hardcoded values**

Interview Questions

- **Why dictionary instead of list?**
- **How to persist data later?**

- **How would this scale?**

➡️ **Outcome: You *think in data structures*.**

WEEK 4 (Mar 3 – Mar 9)

🧠 **Functions, Recursion & Modularity**

🧪 **Project 4: Math Toolkit**

What to build

A menu-based tool:

- **Factorial**
- **Prime check**
- **GCD / LCM**
- **Fibonacci (recursive + iterative)**

Concepts Used

- **Functions**
- **Recursion**
- **Modular design**

Clean Code Checklist

✅ **No duplicated logic**
✅ **Clear base cases**
✅ **Function documentation**
✅ **Predictable output**

Interview Questions

- **Why recursion here?**
- **Stack overflow risks?**
- **Iterative vs recursive tradeoffs?**

➡️ **Outcome: You understand *how Python executes code*.**

WEEK 5 (Mar 10 – Mar 16)

📂 File Handling & Error Handling

🧪 Project 5: File-Based Notes App

What to build

- **Add notes**
- **View notes**
- **Delete notes**
- **Store in `.txt` or `.json`**

Concepts Used

- **File I/O**
- **`try / except`**
- **Data persistence**

Clean Code Checklist

✅ Error handling present
✅ Files safely closed
✅ User-friendly messages
✅ Data format consistency

Interview Questions

- **Why JSON over TXT?**
- **What happens if file is missing?**
- **How would you encrypt notes?**

➡️ **Outcome: You feel like a *real programmer*.**

WEEK 6 (Mar 17 – Mar 23)

🧩 OOP (Only What You Need)

🧪 Project 6: Bank Account Simulator

What to build

- **Create account**
- **Deposit / withdraw**
- **Show balance**
- **Prevent invalid operations**

Concepts Used

- **Classes**
- **Encapsulation**
- **Methods**

Clean Code Checklist

✅ Private variables
✅ Clear class responsibilities
✅ No logic in global scope

Interview Questions

- **Why use OOP here?**
- **How would you add multiple users?**
- **How to prevent misuse?**

➡️ Outcome: You can explain OOP clearly.

WEEK 7 (Mar 24 – Mar 31)

🔥 Final Project (Portfolio Ready)

🧪 Project 7: CLI Task Manager

Features

- Add task
- Mark done
- Filter pending
- Save to file

Interview Power Questions

- How is your code structured?
- How would you add a UI?
- How would you test this?
- How would you scale this?

🧭 TIMELINE SUMMARY

Phase	Date
Python Basics	Feb 10 – Feb 16
Core Logic	Feb 17 – Feb 23
Data Structures	Feb 24 – Mar 2
Functions & Recursion	Mar 3 – Mar 9
Files & Errors	Mar 10 – Mar 16
OOP	Mar 17 – Mar 23
Final Project	Mar 24 – Mar 31

🐍 PYTHON FOR DSA & LEETCODE (COMPLETE ROADMAP)

0️⃣ Python Setup & Mindset (1 Day)

Learn

- Install Python 3.x
- Use VS Code / PyCharm
- Run code from terminal
- `print()`, comments

Solve (Warm-up)

- LC 1929 – Concatenation of Array
- LC 1480 – Running Sum of 1D Array

➡️ **Switch when**: you can run & debug code confidently

1️⃣ Python Basics (Syntax & Flow)

Learn

- Variables & data types (`int`, `float`, `str`, `bool`)
- Input / Output
- Type casting
- Operators
- `if / elif / else`

Solve

- LC 7 – Reverse Integer
- LC 9 – Palindrome Number
- LC 258 – Add Digits
- LC 231 – Power of Two

➡️ **Switch when**: no confusion with indentation & conditions

---

2️⃣ Loops & Logic Building

Learn

- `for`, `while`
- `range()`
- Nested loops
- `break`, `continue`

Solve

- LC 412 – Fizz Buzz
- LC 628 – Maximum Product of Three Numbers
- LC 326 – Power of Three
- LC 342 – Power of Four

➡️ **Switch when**: you think in loops automatically

3️⃣ Strings in Python

Learn

- String indexing & slicing
- Immutable nature
- Common methods (`split`, `join`, `lower`, `count`)
- ASCII values (`ord`, `chr`)

Solve

- LC 344 – Reverse String
- LC 125 – Valid Palindrome
- LC 58 – Length of Last Word
- LC 14 – Longest Common Prefix
- LC 242 – Valid Anagram

➡️ **Switch when**: slicing feels natural

4️⃣ Lists (Arrays in Python)

Learn

- List creation
- Indexing
- List methods (`append`, `pop`, `sort`, `reverse`)
- Shallow vs deep copy

Solve

- LC 217 – Contains Duplicate
- LC 977 – Squares of a Sorted Array
- LC 189 – Rotate Array
- LC 283 – Move Zeroes
- LC 53 – Maximum Subarray

➡️ **Switch when**: you stop brute-forcing

5️⃣ Python Built-in Data Structures

Learn

- `list`
- `set`
- `dict`
- `tuple`
- `collections.Counter`

Solve (Hashing Focus)

- LC 1 – Two Sum
- LC 205 – Isomorphic Strings

- LC 128 – Longest Consecutive Sequence
- LC 560 – Subarray Sum Equals K

➡️ **Switch when**: you instinctively reach for dict/set

6️⃣ Functions & Recursion

Learn

- Function definition
- Return vs print
- Parameters
- Recursion basics & base case

Solve

- LC 509 – Fibonacci Number
- LC 50 – Pow(x, n)
- LC 344 – Reverse String (recursive)
- LC 206 – Reverse Linked List (recursive)

➡️ **Switch when**: base case is obvious to you

7️⃣ Time & Space Complexity (Python Perspective)

Learn

- Big-O notation
- Python operation costs:
  - `list.append()` → O(1)
  - `list.pop()` → O(1)
  - `in` on list → O(n)
  - `in` on set/dict → O(1)

Solve

- LC 121 – Best Time to Buy and Sell Stock
- LC 122 – Buy and Sell Stock II

➡️ **Switch when**: you avoid TLE mentally

🚀 **PYTHON + CORE DSA (LEETCODE)**

From here, Python learning is **done**.
Now Python becomes just a **tool** for DSA.

Binary Search (Python Style)

- LC 704 – Binary Search
- LC 35 – Search Insert Position
- LC 34 – First and Last Position
- LC 153 – Find Min in Rotated Sorted Array
- LC 875 – Koko Eating Bananas

Sliding Window / Two Pointers

- LC 3 – Longest Substring Without Repeating Characters
- LC 424 – Longest Repeating Character Replacement
- LC 904 – Fruit Into Baskets
- LC 76 – Minimum Window Substring

Stack & Queue

Learn

- Stack using list
- Queue using `collections.deque`

Solve

- LC 20 – Valid Parentheses

- LC 155 – Min Stack
- LC 496 – Next Greater Element
- LC 84 – Largest Rectangle in Histogram
- LC 42 – Trapping Rain Water

Heaps (IMPORTANT for Python)

Learn

- `heapq` (min heap)
- Push / pop
- Heap of tuples

Solve

- LC 215 – Kth Largest Element
- LC 347 – Top K Frequent Elements
- LC 295 – Find Median from Data Stream

Trees (Python)

- LC 94 – Inorder Traversal
- LC 102 – Level Order
- LC 104 – Max Depth
- LC 543 – Diameter of Binary Tree
- LC 236 – LCA

Graphs (Python BFS/DFS)

- LC 200 – Number of Islands
- LC 733 – Flood Fill
- LC 994 – Rotten Oranges

- LC 207 – Course Schedule
- LC 127 – Word Ladder

Dynamic Programming (Python Friendly)

Start With

- LC 70 – Climbing Stairs
- LC 198 – House Robber
- LC 322 – Coin Change
- LC 416 – Partition Equal Subset Sum

Advanced

- LC 300 – LIS
- LC 1143 – LCS
- LC 72 – Edit Distance

🧠 **WHEN PYTHON IS "ENOUGH" FOR LEETCODE**

You are **Python-ready** when:

- You write clean loops without thinking
- You use dict/set naturally
- You don't fear recursion
- You solve mediums in ≤45 min

❌ **IGNORE THESE (FOR NOW)**

- OOP depth
- Decorators
- Generators

- Async / await
- Pandas / NumPy